Node.js Authentication App + Mocha/Chai + Jenkins Pipeline

Part 1: Application Setup & Exploration

Step 1.1: Obtain the Node.js Application of your own choice

- Get **Node.js Authentication App (Login/Signup system)**
- Clone or download the repository locally

Step 1.2: Install Dependencies

- Navigate to the project directory
- Install project dependencies using npm
- Install the following as **dev dependencies**:
    - Mocha
    - Chai

Step 1.3: Explore Project Structure

Analyze the architecture:

- src/ → Core application logic
- Authentication modules (Login/Signup handling)
- package.json → Scripts & dependencies
- UI pages for:
    - Login
    - Signup
    - Basic navigation / routing

Step 1.4: Run the Application

- Start the application
- Verify it runs on **localhost:3000**
- Test:
    - Login page
    - Signup page

Checkpoint: Application runs successfully without errors

Part 2: Test Automation (Mocha + Chai + POM)

Step 2.1: Test Structure (POM-Based)

Create a scalable testing structure:

```
tests/
├── unit/
├── integration/
├── pages/        → Page Object Model files
└── utils/        → Helper functions (optional)
```

Step 2.2: Apply Page Object Model (Mandatory)

- Create **Page Object classes** for:
    - Login Page
    - Signup Page

Each page object must encapsulate:

- Element locators (input fields, buttons, messages)
- Actions (enter data, submit forms)
- Validation methods (error/success states)

Step 2.3: Unit Testing (Validation Logic)

Create unit tests for:

Email Validation

- Valid email formats
- Missing @ symbol
- Missing domain
- Empty email

Password Validation

- Valid password (≥ 6 characters)
- Short password (< 6 characters)

Username Validation

- Valid username (≥ 3 characters)
- Short username (< 3 characters)

Step 2.4: Integration Testing (UI Behavior & Form Interaction)

Using **POM-based automation**, design tests that simulate real user interaction:

Login Page Scenarios

- Enter valid credentials → successful login
- Enter wrong password → error message displayed
- Enter non-existing user → validation message
- Submit empty form → required field errors
- Input invalid email format → validation error

Signup Page Scenarios

- Register with valid data → success message
- Submit form with missing fields → validation errors
- Enter short password → error message
- Enter invalid email → validation error
- Enter short username → error message

Advanced Interaction Scenarios

Students must **design additional test cases**, including:

- Automatically filling forms with **incorrect/random inputs**
- Boundary testing (min/max input lengths)
- Special characters in username/email/password
- Rapid multiple submissions (button spam handling)
- UI element identification and interaction (selectors, IDs, classes)

Requirement: Students should actively explore and add **at least 5–8 additional meaningful test cases** beyond the basic ones.

Step 2.5: Test Scripts Configuration

Update package.json with:

- Run all tests
- Run unit tests
- Run integration tests
- Generate test reports

Step 2.6: Local Execution

- Execute all test suites
- Validate:
    - All tests pass
    - Unit & integration tests run independently

Checkpoint: Stable, fully passing test suite

Part 3: Jenkins CI/CD Pipeline

Step 3.1: Launch Jenkins

- Start Jenkins using the WAR file
- Keep terminal active

Step 3.2: Initial Setup

- Access via browser (localhost:8080)
- Unlock Jenkins
- Install suggested plugins
- Create admin user

Step 3.3: Plugin Installation

Install:

- NodeJS Plugin
- HTML Publisher Plugin
- Verify Git Plugin

Step 3.4: Configure Node.js

- Navigate to **Manage Jenkins → Tools**
- Add Node.js (v18.x recommended)
- Enable automatic installation

Step 3.5: Create Jenkins Job

Option A: Freestyle Project

- Configure Git repository
- Setup Node.js environment
- Add build steps:
    - Install dependencies
    - Execute tests
- Publish reports

Option B: Pipeline (Recommended – Industry Standard)

Define stages:

1. **Checkout Code**
2. **Install Dependencies**
3. **Run Unit Tests**
4. **Run Integration Tests**
5. **Generate Reports**

Include **post-build actions**:

- Success handling
- Failure handling

Step 3.6: Execute Pipeline

- Trigger build manually
- Monitor console logs
- Verify:
    - All stages execute successfully
    - Tests pass
    - Reports generated

Checkpoint: Fully functional CI pipeline

Part 4: Demonstration

Requirements

- Duration: 2-**3 (strict)**
- Clear narration

Must Cover:

1. Introduction

- Your name
- Objective of assignment

2. Application Overview

- Folder structure
- Login/Signup functionality

3. Test Architecture

- POM structure
- Unit & integration tests overview

4. Local Execution

- Run tests
- Show passing results

5. Jenkins Pipeline

- Job configuration
- Build execution
- Test results

6. Closing

- Key learnings

Part 5: Documentation

README.md Must Include:

Project Overview

- Description of authentication system

Testing Purpose

- Why automation is implemented

Tech Stack

- Node.js
- Express
- Mocha
- Chai
- Jenkins

Test Summary Table

Type	Count	Status
Unit Tests	X	Pass/Fail
Integration Tests	X	Pass/Fail
Total	X	Pass/Fail

Jenkins Pipeline Explanation

- Stage-wise breakdown

Submission Details

- Video
- GitHub repo

Author Info

- Name
- Submission date

Required Screenshots

- Local test results (all passing)
- Jenkins successful build
- Jenkins test report

Grading Rubric

Section	Points	Criteria
Setup	10	App runs correctly
Testing	40	POM usage + coverage
Jenkins	30	Pipeline execution
Video	10	Clarity + completeness
Documentation	10	Professional README
Total	**100**	

Extra Credit (+10 Points)

- Advanced edge-case testing
- Code coverage integration
- Email notifications in Jenkins
- Multi-branch pipeline